```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta
    name="description"
    content="Delícia Bolos: bolos e doces artesanais feitos sob encomenda com carinho, qualidade e atendimento personalizado."
  />
  <title>Delícia Bolos</title>

  <style>
    :root {
      --bg: #fffafc;
      --bg-soft: #fff3f7;
      --surface: rgba(255, 255, 255, 0.96);
      --surface-strong: #ffffff;
      --primary: #c4517b;
      --primary-dark: #8f2f4f;
      --primary-soft: #f8dce7;
      --secondary: #6b4a56;
      --text: #32232a;
      --text-soft: #5d4952;
      --border: #ecd3dd;
      --success: #067647;
      --success-bg: #ecfdf3;
      --success-border: #abefc6;
      --danger: #b42318;
      --danger-bg: #fef3f2;
      --danger-border: #fecdca;
```

```
  --warning: #b54708;
  --warning-bg: #fffaeb;
  --warning-border: #fedf89;
  --focus: rgba(196, 81, 123, 0.22);
  --shadow-sm: 0 10px 30px rgba(196, 81, 123, 0.08);
  --shadow-md: 0 18px 50px rgba(196, 81, 123, 0.14);
  --radius-sm: 14px;
  --radius-md: 18px;
  --radius-lg: 22px;
  --radius-xl: 26px;
  --max-width: 1200px;

  --space-2: 8px;
  --space-3: 12px;
  --space-4: 16px;
  --space-5: 20px;
  --space-6: 24px;
  --space-7: 28px;
  --space-8: 32px;
}

* {
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
```

```
  font-family: "Inter", "Segoe UI", Arial, sans-serif;
  color: var(--text);
  background:
    radial-gradient(circle at top, #ffe8f1 0%, transparent 28%),
    linear-gradient(180deg, #fffafc 0%, #fff4f8 100%);
}

h1, h2, h3, h4, p, ul {
  margin-top: 0;
}

img {
  max-width: 100%;
  display: block;
}

button, input, select, textarea {
  font: inherit;
}

a {
  color: inherit;
}

a:focus-visible,
button:focus-visible,
input:focus-visible,
select:focus-visible,
textarea:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 2px;
```

```
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}

.container {
  width: min(92%, var(--max-width));
  margin: 0 auto;
}

.mt-5 {
  margin-top: var(--space-5);
}

.header {
  background: linear-gradient(180deg, #cd5a84 0%, #a43b60 100%);
  color: white;
  padding: 76px 0 54px;
  position: relative;
  overflow: hidden;
  box-shadow: 0 16px 45px rgba(0, 0, 0, 0.08);
}
```

```
.hero {
  display: grid;
  grid-template-columns: 1.2fr 0.8fr;
  gap: 28px;
  align-items: center;
}

.hero-badge {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 10px 14px;
  border-radius: 999px;
  border: 1px solid rgba(255, 255, 255, 0.28);
  background: rgba(255, 255, 255, 0.12);
  backdrop-filter: blur(8px);
  font-size: 14px;
  font-weight: 700;
  margin-bottom: 18px;
  color: #fff;
}

.hero-title {
  font-size: clamp(2.4rem, 4vw, 4rem);
  line-height: 1.02;
  letter-spacing: -0.04em;
  margin-bottom: 16px;
}

.hero-description {
```

```
  color: rgba(255, 255, 255, 0.96);
  line-height: 1.8;
  max-width: 700px;
  font-size: 1.06rem;
  margin-bottom: 24px;
}

.hero-actions {
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
}

.btn-primary,
.btn-secondary,
.btn-outline-soft {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-height: 48px;
  padding: 0 18px;
  border-radius: 999px;
  text-decoration: none;
  font-weight: 800;
  transition: 0.2s ease;
  cursor: pointer;
  border: none;
}

.btn-primary {
  background: white;
```

```
  color: var(--primary-dark);
  box-shadow: 0 14px 30px rgba(0, 0, 0, 0.12);
}

.btn-primary:hover {
  transform: translateY(-1px);
}

.btn-secondary {
  color: white;
  border: 1px solid rgba(255, 255, 255, 0.28);
  background: rgba(255, 255, 255, 0.08);
}

.btn-secondary:hover {
  background: rgba(255, 255, 255, 0.14);
}

.hero-panel {
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.16);
  border-radius: var(--radius-xl);
  padding: var(--space-6);
  backdrop-filter: blur(8px);
  box-shadow: 0 18px 40px rgba(0, 0, 0, 0.1);
}

.hero-panel h2 {
  color: white;
  margin-bottom: 14px;
  font-size: 1.1rem;
```

```
}

.hero-panel ul {
  list-style: none;
  padding: 0;
  margin: 0;
  display: grid;
  gap: 12px;
}

.hero-panel li {
  padding: 12px 14px;
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.08);
  color: rgba(255, 255, 255, 0.96);
  line-height: 1.6;
  font-size: 0.97rem;
}

.main-nav {
  position: sticky;
  top: 0;
  z-index: 40;
  background: rgba(255, 247, 250, 0.9);
  backdrop-filter: blur(12px);
  border-bottom: 1px solid rgba(236, 211, 221, 0.9);
}

.nav-inner {
  width: min(94%, var(--max-width));
  margin: 0 auto;
```

```
  padding: 14px 0;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
}

.brand-mini {
  font-weight: 900;
  letter-spacing: -0.03em;
  color: var(--primary-dark);
  text-decoration: none;
}

.nav-links {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}

.nav-links a {
  text-decoration: none;
  color: var(--primary-dark);
  font-weight: 700;
  font-size: 0.96rem;
  padding: 10px 14px;
  border-radius: 999px;
  transition: 0.2s ease;
}

.nav-links a:hover {
```

```
  background: white;
  box-shadow: var(--shadow-sm);
}

main {
  padding: 28px 0 60px;
}

section {
  margin-bottom: 22px;
  scroll-margin-top: 90px;
}

.section-card {
  background: var(--surface);
  border: 1px solid rgba(236, 211, 221, 0.78);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-sm);
  padding: 30px;
}

.section-title {
  text-align: center;
  color: var(--primary-dark);
  font-size: clamp(1.8rem, 2vw, 2.3rem);
  letter-spacing: -0.03em;
  margin-bottom: 10px;
}

.section-subtitle {
  max-width: 860px;
```

```
  margin: 0 auto 22px;
  text-align: center;
  color: var(--text-soft);
  line-height: 1.8;
  font-size: 1.02rem;
}

.grid-3 {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 16px;
}

.info-card,
.catalog-card,
.faq-card,
.contato-card,
.fundador-card {
  background: linear-gradient(180deg, #fffafd 0%, #fff4f8 100%);
  border: 1px solid var(--border);
  border-radius: 20px;
  padding: 20px;
}

.info-card h3,
.catalog-card h3,
.faq-card h3,
.contato-card h3,
.fundador-card h3 {
  color: var(--primary-dark);
  margin-bottom: 8px;
```

```
  font-size: 1.04rem;
}

.info-card p,
.catalog-card p,
.faq-card p,
.contato-card p,
.fundador-card p {
  margin: 0;
  color: var(--text-soft);
  line-height: 1.75;
}

.catalog-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 16px;
}

.catalog-badge {
  display: inline-flex;
  align-items: center;
  min-height: 32px;
  padding: 0 12px;
  border-radius: 999px;
  background: var(--primary-soft);
  color: var(--primary-dark);
  font-size: 0.88rem;
  font-weight: 800;
  margin-bottom: 12px;
}
```

```
.catalog-highlight {
  margin-top: 14px;
  font-weight: 700;
  color: var(--text);
}

.testimonials-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 16px;
}

.testimonial-card {
  background: linear-gradient(180deg, #fffafd 0%, #fff4f8 100%);
  border: 1px solid var(--border);
  border-radius: 20px;
  padding: 20px;
  box-shadow: var(--shadow-sm);
}

.testimonial-card p {
  margin: 0;
  color: var(--text-soft);
  line-height: 1.8;
}

.testimonial-card strong {
  display: block;
  margin-top: 14px;
  color: var(--primary-dark);
```

```
  font-size: 0.98rem;
}

.pedido-layout {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 360px;
  gap: 20px;
  align-items: start;
}

.pedido-main {
  display: grid;
  gap: 18px;
}

.pedido-sidebar {
  position: sticky;
  top: 88px;
}

.bloco {
  background: white;
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding: 22px;
  box-shadow: var(--shadow-sm);
}

.bloco h3 {
  color: var(--primary-dark);
  font-size: 1.25rem;
```

```
  margin-bottom: 8px;
}

.bloco-desc {
  color: var(--text-soft);
  line-height: 1.7;
  margin-bottom: 18px;
}

.produtos-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
}

.produto-card {
  border: 1px solid var(--border);
  border-radius: 18px;
  background: linear-gradient(180deg, #fffdfd 0%, #fff7fa 100%);
  padding: 16px;
  display: flex;
  flex-direction: column;
  gap: 12px;
  min-height: 100%;
}

.produto-topo {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  gap: 12px;
```

```
}

.produto-topo h4 {
  margin: 0;
  font-size: 1.02rem;
  color: var(--text);
  line-height: 1.35;
}

.produto-meta {
  color: var(--text-soft);
  font-size: 0.95rem;
  line-height: 1.6;
}

.produto-actions {
  display: grid;
  gap: 10px;
  margin-top: auto;
}

.produto-inline {
  display: grid;
  grid-template-columns: 1fr 110px;
  gap: 10px;
  align-items: end;
}

.field {
  display: block;
}
```

```
.field label,
.fieldset-title,
.field legend {
  display: block;
  margin-bottom: 8px;
  color: var(--primary-dark);
  font-size: 0.94rem;
  font-weight: 800;
}

input,
select,
textarea {
  width: 100%;
  min-height: 46px;
  border-radius: 13px;
  border: 1px solid var(--border);
  background: white;
  color: var(--text);
  padding: 10px 12px;
  outline: none;
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

textarea {
  min-height: 110px;
  resize: vertical;
}

input:focus,
```

```
select:focus,
textarea:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 4px rgba(196, 81, 123, 0.12);
}

input[type="radio"] {
  width: auto;
  min-height: auto;
  margin: 0;
  accent-color: var(--primary);
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
}

.full {
  grid-column: 1 / -1;
}

.radio-group {
  border: 0;
  padding: 0;
  margin: 18px 0 0;
  min-width: 0;
}

.inline-options {
```

```
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
}

.inline-options label {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  border: 1px solid var(--border);
  border-radius: 999px;
  padding: 12px 14px;
  background: #fff;
  font-weight: 700;
  cursor: pointer;
  transition: 0.2s ease;
}

.inline-options label:hover {
  background: #fff7fa;
  border-color: #e7b5c8;
  transform: translateY(-1px);
}

.brigadeiros-topo {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 14px;
  align-items: end;
  margin-bottom: 16px;
}
```

```
.status-box,
.help-box {
  padding: 12px 14px;
  border-radius: 14px;
  font-size: 0.95rem;
  line-height: 1.6;
}

.help-box {
  background: var(--warning-bg);
  border: 1px solid var(--warning-border);
  color: var(--warning);
  font-weight: 700;
  text-align: center;
  min-width: 220px;
}

.brigadeiros-lista {
  display: grid;
  grid-template-columns: repeat(3, minmax(220px, 1fr));
  gap: 12px;
}

.brigadeiros-lista label {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 12px 14px;
  background: #fffafc;
```

```
  border: 1px solid var(--border);
  border-radius: 14px;
  font-weight: 700;
}

.brigadeiros-lista input[type="number"] {
  width: 88px;
  min-width: 88px;
  text-align: center;
}

#brigadeiroStatus {
  margin-top: 16px;
}

.status-neutral {
  background: #fff6fa;
  border: 1px dashed #e5b5c7;
  color: var(--primary-dark);
  font-weight: 700;
}

.status-success {
  background: var(--success-bg);
  border: 1px solid var(--success-border);
  color: var(--success);
  font-weight: 700;
}

.status-warning {
  background: var(--warning-bg);
```

```
  border: 1px solid var(--warning-border);
  color: var(--warning);
  font-weight: 700;
}

.status-danger {
  background: var(--danger-bg);
  border: 1px solid var(--danger-border);
  color: var(--danger);
  font-weight: 700;
}

.sidebar-card {
  background: var(--surface-strong);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-md);
  padding: 22px;
}

.sidebar-card h3 {
  color: var(--primary-dark);
  margin-bottom: 12px;
}

.sidebar-text {
  color: var(--text-soft);
  line-height: 1.7;
  font-size: 0.95rem;
}
```

```
.sidebar-divider {
  height: 1px;
  background: var(--border);
  margin: 16px 0;
}

.total-box {
  background: linear-gradient(180deg, #fff8fb 0%, #fff1f7 100%);
  border: 1px solid var(--border);
  border-radius: 18px;
  padding: 18px;
  text-align: center;
  margin-top: 16px;
}

.total-box strong {
  display: block;
  font-size: 1.7rem;
  color: var(--primary-dark);
  letter-spacing: -0.03em;
  margin-bottom: 10px;
}

.total-secundario {
  color: var(--text);
  font-size: 1rem;
  font-weight: 700;
  margin-top: 8px;
}

.resumo-visual {
```

```
  display: grid;
  gap: 10px;
  margin-top: 16px;
}

.resumo-vazio {
  border: 1px dashed var(--border);
  border-radius: 16px;
  padding: 14px;
  color: var(--text-soft);
  background: #fffafc;
  line-height: 1.7;
  text-align: center;
}

.resumo-item {
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 12px 14px;
  background: #fff;
}

.resumo-item strong {
  display: block;
  color: var(--text);
  margin-bottom: 4px;
  font-size: 0.97rem;
}

.resumo-item span {
  color: var(--text-soft);
```

```
  font-size: 0.93rem;
  line-height: 1.6;
}

.resumo-total-linha {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  color: var(--text);
  font-weight: 700;
  font-size: 0.96rem;
}

.form-message {
  margin-top: 14px;
  padding: 12px 14px;
  border-radius: 14px;
  font-weight: 700;
  line-height: 1.6;
}

.hidden {
  display: none !important;
}

.button-primary {
  width: 100%;
  min-height: 52px;
  border: none;
  border-radius: 14px;
  background: linear-gradient(180deg, var(--primary) 0%, var(--primary-dark) 100%);
```

```
  color: white;
  font-size: 1rem;
  font-weight: 900;
  cursor: pointer;
  box-shadow: 0 14px 28px rgba(196, 81, 123, 0.24);
  transition: 0.2s ease;
}

.button-primary:hover {
  transform: translateY(-1px);
  box-shadow: 0 18px 32px rgba(196, 81, 123, 0.28);
}

.button-primary:disabled {
  opacity: 0.58;
  cursor: not-allowed;
  transform: none;
  box-shadow: none;
}

.contato-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 16px;
  max-width: 980px;
  margin: 0 auto;
}

.contato-card {
  text-align: center;
  box-shadow: var(--shadow-sm);
```

```
  background: #fff;
}

.contato-card a {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-height: 44px;
  padding: 0 16px;
  border-radius: 999px;
  text-decoration: none;
  font-weight: 800;
  background: var(--bg-soft);
  border: 1px solid var(--border);
  color: var(--primary-dark);
  transition: 0.2s ease;
  margin-top: 14px;
}

.contato-card a:hover {
  transform: translateY(-1px);
  background: white;
}

.mapa-card {
  margin-top: 18px;
  border: 1px solid var(--border);
  border-radius: 20px;
  background: #fff;
  padding: 20px;
  box-shadow: var(--shadow-sm);
```

```
}

.mapa-card h3 {
  color: var(--primary-dark);
  margin-bottom: 8px;
}

.mapa-card p {
  color: var(--text-soft);
  line-height: 1.7;
  margin-bottom: 14px;
}

.mapa-embed {
  overflow: hidden;
  border-radius: 18px;
  border: 1px solid var(--border);
}

.mapa-embed iframe {
  display: block;
  width: 100%;
  height: 360px;
  border: 0;
}

.fundadores-lista {
  list-style: none;
  padding: 0;
  margin: 0;
  display: grid;
```

```
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 12px;
}

.fundador-card {
  text-align: center;
  font-weight: 800;
}

.faq-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16px;
}

.erro-campo {
  border-color: var(--danger) !important;
  box-shadow: 0 0 0 4px rgba(180, 35, 24, 0.1) !important;
}

.footer {
  margin-top: 36px;
  background: linear-gradient(180deg, var(--primary) 0%, var(--primary-dark) 100%);
  color: white;
  text-align: center;
  padding: 28px 20px;
}

.footer p {
  margin: 0;
  color: rgba(255, 255, 255, 0.95);
```

```
}

@media (max-width: 1100px) {
  .pedido-layout {
    grid-template-columns: 1fr;
  }

  .pedido-sidebar {
    position: static;
  }

  .hero {
    grid-template-columns: 1fr;
  }

  .fundadores-lista {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}

@media (max-width: 900px) {
  .grid-3,
  .catalog-grid,
  .faq-grid,
  .contato-grid,
  .produtos-grid,
  .testimonials-grid {
    grid-template-columns: 1fr;
  }

  .fundadores-lista {
```

```
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .brigadeiros-lista {
    grid-template-columns: repeat(2, minmax(220px, 1fr));
  }

  .form-grid {
    grid-template-columns: 1fr;
  }

  .full {
    grid-column: auto;
  }
}

@media (max-width: 720px) {
  .nav-inner {
    flex-direction: column;
    align-items: flex-start;
  }

  .hero-actions {
    flex-direction: column;
  }

  .btn-primary,
  .btn-secondary {
    width: 100%;
  }
```

```
    .brigadeiros-topo {
      grid-template-columns: 1fr;
    }
  }

  @media (max-width: 640px) {
    .header {
      padding: 56px 0 40px;
    }

    .section-card,
    .bloco,
    .sidebar-card,
    .mapa-card {
      padding: 18px;
    }

    .produto-inline {
      grid-template-columns: 1fr;
    }

    .brigadeiros-lista,
    .fundadores-lista {
      grid-template-columns: 1fr;
    }

    .mapa-embed iframe {
      height: 280px;
    }
  }
</style>
```

```
</head>
<body>
  <a href="#conteudo" class="sr-only">Pular para o conteúdo principal</a>

  <header class="header">
    <div class="container hero">
      <div>
        <div class="hero-badge">Encomendas com antecedência mínima de 1 dia</div>
        <h1 class="hero-title">Delícia Bolos</h1>
        <p class="hero-description">
          Bolos e doces artesanais feitos sob encomenda para aniversários, festas,
          cafés especiais e momentos inesquecíveis.
        </p>

        <div class="hero-actions">
          <a href="#pedido" class="btn-primary">Fazer pedido</a>
          <a href="#contato" class="btn-secondary">Falar conosco</a>
        </div>
      </div>

      <aside class="hero-panel" aria-label="Informações importantes">
        <h2>Informações importantes</h2>
        <ul>
          <li>Pedidos com no mínimo 1 dia de antecedência.</li>
          <li>Horário disponível para pedidos e retirada entre 07:00 e 20:00.</li>
          <li>Pagamento à vista ou 50% de sinal + 50% no recebimento.</li>
          <li>Brigadeiros com pedido mínimo de 50 unidades.</li>
        </ul>
      </aside>
    </div>
  </header>
```

```
<nav class="main-nav" aria-label="Navegação principal">
  <div class="nav-inner">
    <a href="#" class="brand-mini">Delícia Bolos</a>

    <div class="nav-links">
      <a href="#sobre">Sobre</a>
      <a href="#produtos">Produtos</a>
      <a href="#depoimentos">Depoimentos</a>
      <a href="#pedido">Pedido</a>
      <a href="#faq">Dúvidas</a>
      <a href="#contato">Contato</a>
    </div>
  </div>
</nav>

<main id="conteudo" class="container">
  <section id="sobre" class="section-card" aria-labelledby="sobre-title">
    <h2 id="sobre-title" class="section-title">Sobre a Delícia Bolos</h2>
    <p class="section-subtitle">
      Há mais de 10 anos no ramo da confeitaria, a Delícia Bolos transforma celebrações em experiências
      cheias de sabor, delicadeza e qualidade. Cada encomenda é feita com atenção aos detalhes,
      combinando beleza, textura e um sabor que fica na memória.
    </p>

    <div class="grid-3">
      <article class="info-card">
        <h3>Ingredientes selecionados</h3>
        <p>
```

Receitas preparadas com cuidado, ingredientes de qualidade e acabamento pensado para impressionar.

</article>

<article class="info-card">

<h3>Atendimento personalizado</h3>

Cada pedido é acompanhado com atenção para garantir mais segurança, organização e satisfação.

</article>

<article class="info-card">

<h3>Retirada ou entrega</h3>

Escolha a melhor opção para você e informe o endereço caso deseje entrega.

</article>

</section>

<section id="produtos" class="section-card" aria-labelledby="produtos-title">

<h2 id="produtos-title" class="section-title">Nossos Produtos</h2>

Escolha entre bolos gourmets, bolos simples e brigadeiros artesanais preparados com muito carinho.

<article class="catalog-card">

Bolos Gourmets

```
    <h3>Sabores especiais por quilo</h3>
    <p>
      Opções mais elaboradas com combinações como Ninho com Nutella, Pistache,
      Sensação, Floresta Negra e outras versões para ocasiões especiais.
    </p>
    <div class="catalog-highlight">A partir de R$ 55,00 por kg</div>
  </article>

  <article class="catalog-card">
    <div class="catalog-badge">Bolos Simples</div>
    <h3>Perfeitos para o dia a dia</h3>
    <p>
      Sabores clássicos como cenoura, milho, laranja, formigueiro, bolo de leite
      e outras opções para café da tarde e encomendas rápidas.
    </p>
    <div class="catalog-highlight">A partir de R$ 25,00 por unidade</div>
  </article>

  <article class="catalog-card">
    <div class="catalog-badge">Brigadeiros</div>
    <h3>Caixinhas e festas</h3>
    <p>
      Monte seu pedido escolhendo a quantidade total e distribuindo entre sabores
      variados como beijinho, churros, pistache, leite ninho e muito mais.
    </p>
    <div class="catalog-highlight">R$ 1,40 por unidade • mínimo de 50</div>
  </article>
 </div>
</section>

<section id="depoimentos" class="section-card" aria-labelledby="depoimentos-title">
```

```
<h2 id="depoimentos-title" class="section-title">Depoimentos</h2>
<p class="section-subtitle">
  Veja o que nossos clientes dizem sobre os bolos e doces da Delícia Bolos.
</p>

<div class="testimonials-grid">
  <article class="testimonial-card">
    <p>
      “O bolo ficou simplesmente maravilhoso, muito bonito e extremamente saboroso.
      Todo mundo elogiou na festa!”
    </p>
    <strong>Mariana Souza</strong>
  </article>

  <article class="testimonial-card">
    <p>
      “Os brigadeiros vieram impecáveis, super bem feitos e deliciosos.
      Atendimento muito atencioso do começo ao fim.”
    </p>
    <strong>Fernanda Lima</strong>
  </article>

  <article class="testimonial-card">
    <p>
      “Já pedi mais de uma vez e sempre vem tudo perfeito.
      Dá para sentir o carinho em cada detalhe.”
    </p>
    <strong>Juliana Martins</strong>
  </article>
</div>
</section>
```

```
<form id="pedidoForm" novalidate>
  <section id="pedido" class="section-card" aria-labelledby="pedido-title">
    <h2 id="pedido-title" class="section-title">Monte seu pedido</h2>
    <p class="section-subtitle">
      Selecione os itens desejados, informe seus dados e envie sua encomenda de forma simples.
    </p>

    <div class="pedido-layout">
      <div class="pedido-main">
        <section class="bloco" aria-labelledby="gourmets-title">
          <h3 id="gourmets-title">Bolos Gourmets</h3>
          <p class="bloco-desc">
            Escolha o sabor, a massa e a quantidade em quilos.
          </p>

          <div class="produtos-grid" id="bolosGourmetsGrid"></div>
        </section>

        <section class="bloco" aria-labelledby="simples-title">
          <h3 id="simples-title">Bolos Simples</h3>
          <p class="bloco-desc">
            Sabores clássicos para o dia a dia, café da tarde e encomendas especiais.
          </p>

          <div class="produtos-grid" id="bolosSimplesGrid"></div>
        </section>

        <section class="bloco" aligned="brigadeiros-title">
          <h3 id="brigadeiros-title">Brigadeiros</h3>
```

```
<p class="bloco-desc">
  Informe a quantidade total e distribua os sabores conforme desejar.
</p>

<div class="brigadeiros-topo">
  <div class="field">
    <label for="brigadeiroQtd">Quantidade total de brigadeiros</label>
    <input
      type="number"
      id="brigadeiroQtd"
      min="50"
      step="1"
      placeholder="Digite a quantidade total (mínimo 50)"
      aria-describedby="brigadeiroAjuda brigadeiroStatus"
    />
  </div>

  <div id="brigadeiroAjuda" class="help-box">
    Pedido mínimo de 50 unidades
  </div>
</div>

<div class="brigadeiros-lista">
  <label>Amendoim <input type="number" class="brigadeiro-qtd" data-sabor="Amendoim" min="0" value="0" /></label>
  <label>Beijinho <input type="number" class="brigadeiro-qtd" data-sabor="Beijinho" min="0" value="0" /></label>
  <label>Bicho de Pé <input type="number" class="brigadeiro-qtd" data-sabor="Bicho de Pé" min="0" value="0" /></label>
  <label>Brigadeiro <input type="number" class="brigadeiro-qtd" data-sabor="Brigadeiro" min="0" value="0" /></label>
  <label>Brigadeiro de Limão <input type="number" class="brigadeiro-qtd" data-sabor="Brigadeiro de Limão" min="0" value="0" /></label>
```

```
<label>Brigadeiro de Maracujá <input type="number" class="brigadeiro-qtd" data-sabor="Brigadeiro de Maracujá" min="0" value="0" /></label>

<label>Brigadeiro de Ovomaltine <input type="number" class="brigadeiro-qtd" data-sabor="Brigadeiro de Ovomaltine" min="0" value="0" /></label>

<label>Brigadeiro de Paçoca <input type="number" class="brigadeiro-qtd" data-sabor="Brigadeiro de Paçoca" min="0" value="0" /></label>

<label>Casadinho <input type="number" class="brigadeiro-qtd" data-sabor="Casadinho" min="0" value="0" /></label>

<label>Cajuzinho <input type="number" class="brigadeiro-qtd" data-sabor="Cajuzinho" min="0" value="0" /></label>

<label>Churros <input type="number" class="brigadeiro-qtd" data-sabor="Churros" min="0" value="0" /></label>

<label>Ferrero Rocher <input type="number" class="brigadeiro-qtd" data-sabor="Ferrero Rocher" min="0" value="0" /></label>

<label>Leite Ninho <input type="number" class="brigadeiro-qtd" data-sabor="Leite Ninho" min="0" value="0" /></label>

<label>Leite Ninho com Nutella <input type="number" class="brigadeiro-qtd" data-sabor="Leite Ninho com Nutella" min="0" value="0" /></label>

<label>M&amp;amp;M <input type="number" class="brigadeiro-qtd" data-sabor="M&amp;amp;M" min="0" value="0" /></label>

<label>Pistache <input type="number" class="brigadeiro-qtd" data-sabor="Pistache" min="0" value="0" /></label>

<label>Sensação <input type="number" class="brigadeiro-qtd" data-sabor="Sensação" min="0" value="0" /></label>

<label>Surpresa de Uva <input type="number" class="brigadeiro-qtd" data-sabor="Surpresa de Uva" min="0" value="0" /></label>

<label>Romeu e Julieta <input type="number" class="brigadeiro-qtd" data-sabor="Romeu e Julieta" min="0" value="0" /></label>

<label>Napolitano <input type="number" class="brigadeiro-qtd" data-sabor="Napolitano" min="0" value="0" /></label>

<label>Coco Queimado <input type="number" class="brigadeiro-qtd" data-sabor="Coco Queimado" min="0" value="0" /></label>

<label>Brigadeiro com Morango <input type="number" class="brigadeiro-qtd" data-sabor="Brigadeiro com Morango" min="0" value="0" /></label>

<label>Chocolate Meio Amargo <input type="number" class="brigadeiro-qtd" data-sabor="Chocolate Meio Amargo" min="0" value="0" /></label>

<label>Casadinho Sensação <input type="number" class="brigadeiro-qtd" data-sabor="Casadinho Sensação" min="0" value="0" /></label>
```

```
  </div>

  <div id="brigadeiroStatus" class="status-box status-neutral" aria-live="polite">
    Informe a quantidade total para começar a distribuir os sabores.
  </div>
</section>

<section class="bloco" aria-labelledby="dados-title">
  <h3 id="dados-title">Dados do Pedido</h3>
  <p class="bloco-desc">
    Preencha seus dados para concluir o envio da encomenda.
  </p>

  <div class="form-grid">
    <div class="field full">
      <label for="nome">Nome completo</label>
      <input
        type="text"
        id="nome"
        placeholder="Digite seu nome completo"
        required
        autocomplete="name"
      />
    </div>

    <div class="field full">
      <label for="email">E-mail</label>
      <input
        type="email"
        id="email"
        placeholder="Digite seu e-mail"
```

```
    required
    autocomplete="email"
  />
</div>

<fieldset class="radio-group full" aria-describedby="recebimentoAjuda">
  <legend class="fieldset-title">Forma de recebimento</legend>

  <div class="inline-options">
    <label>
      <input type="radio" name="recebimento" value="Retirada" checked />
      Retirada
    </label>
    <label>
      <input type="radio" name="recebimento" value="Entrega" />
      Entrega
    </label>
  </div>

  <div id="recebimentoAjuda" class="status-box status-neutral mt-5">
    Se escolher entrega, informe o endereço completo.
  </div>
</fieldset>

<div class="field">
  <label for="cep">CEP</label>
  <input
    type="text"
    id="cep"
    placeholder="Digite o CEP"
    maxlength="9"
```

```
    autocomplete="postal-code"
  />
</div>

<div class="field full">
  <label for="endereco">Endereço</label>
  <input
    type="text"
    id="endereco"
    placeholder="Rua, avenida, etc."
    autocomplete="street-address"
  />
</div>

<div class="field">
  <label for="numero">Número</label>
  <input
    type="text"
    id="numero"
    placeholder="Número"
    autocomplete="address-line2"
  />
</div>

<div class="field">
  <label for="complemento">Complemento</label>
  <input
    type="text"
    id="complemento"
    placeholder="Apto, bloco, referência..."
    autocomplete="address-line2"
```

```
    />
  </div>

  <div class="field">
    <label for="dataPedido">Data</label>
    <input type="date" id="dataPedido" required />
  </div>

  <div class="field">
    <label for="horaPedido">Horário</label>
    <input type="time" id="horaPedido" required min="07:00" max="20:00"
value="07:00" />
  </div>

  <div class="field full">
    <label for="observacoes">Observações do pedido</label>
    <textarea
      id="observacoes"
      placeholder="Ex.: escrever nome no bolo, preferência de decoração, detalhes do
pedido, etc."
    ></textarea>
  </div>
</div>

<fieldset class="radio-group" aria-describedby="textoCondicaoPagamento">
  <legend class="fieldset-title">Condição de pagamento</legend>

  <div class="inline-options">
    <label>
      <input type="radio" name="condicaoPagamento" value="avista" checked />
      À vista
    </label>
```

```
            <label>
              <input type="radio" name="condicaoPagamento" value="sinal" />
              50% de sinal + 50% no recebimento
            </label>
          </div>
        </fieldset>

        <fieldset class="radio-group" aria-describedby="pagamentoErro">
          <legend class="fieldset-title">Forma de pagamento</legend>

          <div class="inline-options">
            <label><input type="radio" name="pagamento" value="Cartão de crédito" />Cartão
de crédito</label>
            <label><input type="radio" name="pagamento" value="Cartão de débito" />Cartão
de débito</label>
            <label><input type="radio" name="pagamento" value="Pix" />Pix</label>
          </div>

          <div id="textoCondicaoPagamento" class="status-box status-neutral mt-5">
            Pagamento à vista no momento do pedido.
          </div>

          <div id="pagamentoErro" class="form-message status-danger hidden" aria-
live="polite">
            Selecione a forma de pagamento.
          </div>
        </fieldset>

        <div id="formMensagem" class="form-message hidden" aria-live="polite"></div>
      </section>
    </div>
```

```
<aside class="pedido-sidebar" aria-label="Resumo do pedido">
  <div class="sidebar-card">
    <h3>Resumo do pedido</h3>
    <p class="sidebar-text">
      O valor total é atualizado automaticamente conforme você escolhe os itens.
    </p>

    <div class="sidebar-divider"></div>

    <div class="sidebar-text" id="resumoRecebimentoTexto">
      Retirada ou entrega • Pedidos com antecedência mínima de 1 dia • Horário entre 07:00 e 20:00
    </div>

    <div class="total-box">
      <strong>Total: R$ <span id="total">0,00</span></strong>

      <div class="total-secundario" id="linhaEntrada">
        Pagar agora: R$ <span id="sinal">0,00</span>
      </div>

      <div class="total-secundario hidden" id="linhaRestante">
        Restante no recebimento: R$ <span id="restante">0,00</span>
      </div>
    </div>

    <div id="resumoVisual" class="resumo-visual">
      <div class="resumo-vazio">
        Seu pedido ainda está vazio.<br />
        Escolha bolos ou brigadeiros para ver o resumo aqui.
      </div>
```

```
          </div>

          <div class="mt-5">
            <button type="submit" class="button-primary" disabled>
              Enviar Pedido
            </button>
          </div>
        </div>
      </aside>
    </div>
  </section>
</form>

<section id="faq" class="section-card" aria-labelledby="faq-title">
  <h2 id="faq-title" class="section-title">Dúvidas Frequentes</h2>
  <p class="section-subtitle">
    Algumas informações importantes antes de finalizar sua encomenda.
  </p>

  <div class="faq-grid">
    <article class="faq-card">
      <h3>Qual a antecedência mínima?</h3>
      <p>
        Os pedidos devem ser feitos com pelo menos 1 dia de antecedência para garantir
organização e qualidade.
      </p>
    </article>

    <article class="faq-card">
      <h3>Como funciona o pagamento?</h3>
      <p>
```

Você pode pagar à vista no momento do pedido ou escolher a opção com 50% de sinal e 50% no recebimento.

</article>

<article class="faq-card">

<h3>Posso escolher entrega?</h3>

Sim. Você pode escolher retirada ou entrega e informar seu endereço completo no formulário.

</article>

<article class="faq-card">

<h3>Brigadeiros têm mínimo?</h3>

Sim. O pedido mínimo é de 50 unidades, e a distribuição entre sabores precisa somar exatamente a quantidade informada.

</article>

</section>

<section id="contato" class="section-card" aria-labelledby="contato-title">

<h2 id="contato-title" class="section-title">Contato e Localização</h2>

Entre em contato para dúvidas, encomendas e retirada no local.

<article class="contato-card">

<h3>Instagram</h3>

```
      <p>
        Acompanhe novidades, promoções e inspirações da Delícia Bolos.
      </p>
      <a
href="https://www.instagram.com/deliciasbolosbr?igsh=YTR1eHE2MDJzOHJy&utm_source=qr
"
        target="_blank"
        rel="noopener noreferrer"
      >
        Acessar Instagram
      </a>
    </article>

    <article class="contato-card">
      <h3>WhatsApp</h3>
      <p>
        Fale conosco diretamente para dúvidas rápidas e atendimento mais próximo.
      </p>
      <a
        href="https://w.app/bhse5e"
        target="_blank"
        rel="noopener noreferrer"
      >
        Chamar no WhatsApp
      </a>
    </article>

    <article class="contato-card">
      <h3>Endereço</h3>
      <p>
        R. Atlântica - Jardim do Mar,<br />
```

```
        São Bernardo do Campo - SP,<br />
        09750-480
      </p>
      <a
        href="https://www.google.com/maps?q=R.%20Atl%C3%A2ntica%20-%20Jardim%20do%20Mar,%20S%C3%A3o%20Bernardo%20do%20Campo%20-%20SP,%2009750-480"
        target="_blank"
        rel="noopener noreferrer"
      >
        Abrir no mapa
      </a>
    </article>
  </div>

  <div class="mapa-card">
    <h3>Como chegar</h3>
    <p>
      Veja nossa localização no mapa para facilitar sua retirada.
    </p>

    <div class="mapa-embed">
      <iframe
        src="https://www.google.com/maps?q=R.%20Atl%C3%A2ntica%20-%20Jardim%20do%20Mar,%20S%C3%A3o%20Bernardo%20do%20Campo%20-%20SP,%2009750-480&output=embed"
        loading="lazy"
        allowfullscreen
        referrerpolicy="no-referrer-when-downgrade"
        title="Mapa da Delícia Bolos"
      ></iframe>
    </div>
```

```
  </div>
</section>

<section id="fundadores" class="section-card" aria-labelledby="fundadores-title">
  <h2 id="fundadores-title" class="section-title">Fundadores</h2>
  <p class="section-subtitle">
    Conheça os responsáveis pela Delícia Bolos.
  </p>

  <ul class="fundadores-lista">
    <li class="fundador-card">
      <h3>Amanda Oliveira</h3>
      <p>Co-fundadora</p>
    </li>
    <li class="fundador-card">
      <h3>Rodrigo Yudi</h3>
      <p>Co-fundador</p>
    </li>
    <li class="fundador-card">
      <h3>Vinícius Levi</h3>
      <p>Co-fundador</p>
    </li>
    <li class="fundador-card">
      <h3>Paulo Henrique</h3>
      <p>Co-fundador</p>
    </li>
    <li class="fundador-card">
      <h3>Weslley Nascimento</h3>
      <p>Co-fundador</p>
    </li>
  </ul>
```

```
  </section>
</main>

<footer class="footer">
  <p>© 2026 Delícia Bolos — Todos os direitos reservados.</p>
</footer>

<script>
  const FORMSPREE_URL = "https://formspree.io/f/myknqvaj";

  const bolosGourmets = [
    { id: "bolo1", nome: "Brigadeiro", preco: 70, massaId: "massa1" },
    { id: "bolo2", nome: "Morango com Nutella", preco: 75, massaId: "massa2" },
    { id: "bolo3", nome: "Ninho com Nutella", preco: 75, massaId: "massa3" },
    { id: "bolo4", nome: "Pistache", preco: 90, massaId: "massa4" },
    { id: "bolo5", nome: "Brigadeiro com Mousse Maracujá", preco: 75, massaId: "massa5" },
    { id: "bolo6", nome: "Ninho com Uva Verde", preco: 70, massaId: "massa6" },
    { id: "bolo7", nome: "Chocolate com Uva Verde", preco: 80, massaId: "massa7" },
    { id: "bolo8", nome: "Kit Kat", preco: 88, massaId: "massa8" },
    { id: "bolo9", nome: "Sensação", preco: 77, massaId: "massa9" },
    { id: "bolo10", nome: "Ninho com Morango", preco: 80, massaId: "massa10" },
    { id: "bolo11", nome: "Floresta Negra", preco: 80, massaId: "massa11" },
    { id: "bolo12", nome: "Chocolate com Morango", preco: 60, massaId: "massa12" },
    { id: "bolo13", nome: "Abacaxi com Doce de Leite", preco: 60, massaId: "massa13" },
    { id: "bolo14", nome: "Abacaxi com Coco", preco: 60, massaId: "massa14" },
    { id: "bolo15", nome: "Prestígio", preco: 60, massaId: "massa15" },
    { id: "bolo16", nome: "Doce de Leite", preco: 55, massaId: "massa16" }
  ];

  const bolosSimples = [
    { id: "cenoura", nome: "Cenoura", preco: 35 },
```

```
  { id: "milho", nome: "Milho", preco: 30 },
  { id: "laranja", nome: "Laranja", preco: 25 },
  { id: "fuba", nome: "Fubá", preco: 35 },
  { id: "formigueiro", nome: "Formigueiro", preco: 30 },
  { id: "banana", nome: "Banana", preco: 25 },
  { id: "leite", nome: "Bolo de Leite", preco: 35 },
  { id: "boloGeladoCoco", nome: "Bolo Gelado de Coco", preco: 45 },
  { id: "boloAipim", nome: "Bolo de Aipim", preco: 35 },
  { id: "boloLimao", nome: "Bolo de Limão", preco: 45 }
];

const elements = {
  form: document.getElementById("pedidoForm"),
  gourmetGrid: document.getElementById("bolosGourmetsGrid"),
  simplesGrid: document.getElementById("bolosSimplesGrid"),
  brigadeiroQtd: document.getElementById("brigadeiroQtd"),
  brigadeiroStatus: document.getElementById("brigadeiroStatus"),
  nome: document.getElementById("nome"),
  email: document.getElementById("email"),
  cep: document.getElementById("cep"),
  endereco: document.getElementById("endereco"),
  numero: document.getElementById("numero"),
  complemento: document.getElementById("complemento"),
  dataPedido: document.getElementById("dataPedido"),
  horaPedido: document.getElementById("horaPedido"),
  observacoes: document.getElementById("observacoes"),
  pagamentoErro: document.getElementById("pagamentoErro"),
  formMensagem: document.getElementById("formMensagem"),
  total: document.getElementById("total"),
  linhaEntrada: document.getElementById("linhaEntrada"),
  linhaRestante: document.getElementById("linhaRestante"),
```

```
  textoCondicaoPagamento: document.getElementById("textoCondicaoPagamento"),
  resumoVisual: document.getElementById("resumoVisual"),
  resumoRecebimentoTexto: document.getElementById("resumoRecebimentoTexto")
};

function formatarMoeda(valor) {
  return valor.toFixed(2).replace(".", ",");
}

function emailValido(email) {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function getDataMinima() {
  const hoje = new Date();
  hoje.setHours(0, 0, 0, 0);
  hoje.setDate(hoje.getDate() + 1);

  const ano = hoje.getFullYear();
  const mes = String(hoje.getMonth() + 1).padStart(2, "0");
  const dia = String(hoje.getDate()).padStart(2, "0");

  return `${ano}-${mes}-${dia}`;
}

function getCondicaoPagamento() {
  const selecionado =
document.querySelector('input[name="condicaoPagamento"]:checked');
  return selecionado ? selecionado.value : "avista";
}
```

```
function getFormaPagamento() {
  const selecionado = document.querySelector('input[name="pagamento"]:checked');
  return selecionado ? selecionado.value : "";
}

function getFormaRecebimento() {
  const selecionado = document.querySelector('input[name="recebimento"]:checked');
  return selecionado ? selecionado.value : "Retirada";
}

function calcularSubtotalBoloGourmet(id, preco) {
  const quantidade = Math.max(0, parseFloat(document.getElementById(id).value) || 0);
  return quantidade * preco;
}

function calcularSubtotalBoloSimples(id, preco) {
  const quantidade = Math.max(0, parseInt(document.getElementById(id).value, 10) || 0);
  return quantidade * preco;
}

function obterInfoBrigadeiro() {
  const quantidade = Math.max(0, Number(elements.brigadeiroQtd.value) || 0);

  if (quantidade > 0 && quantidade < 50) {
    return { quantidade, preco: 0, invalido: true };
  }

  const precoUnitario = 1.4;
  const preco = quantidade >= 50 ? quantidade * precoUnitario : 0;

  return { quantidade, preco, invalido: false };
```

```
}

function obterDistribuicaoBrigadeiros() {
  const inputs = document.querySelectorAll(".brigadeiro-qtd");
  const sabores = [];
  let total = 0;

  inputs.forEach((input) => {
    const quantidade = Math.max(0, Number(input.value) || 0);
    const sabor = input.dataset.sabor;

    if (quantidade > 0) {
      sabores.push({ sabor, quantidade });
      total += quantidade;
    }
  });

  return { sabores, total };
}

function renderizarProdutos() {
  elements.gourmetGrid.innerHTML = bolosGourmets.map((bolo) => `
    <article class="produto-card">
      <div class="produto-topo">
        <h4>${bolo.nome}</h4>
      </div>
      <div class="produto-meta">R$ ${formatarMoeda(bolo.preco)} por kg</div>

      <div class="produto-actions">
        <div class="produto-inline">
          <div class="field">
```

```
        <label for="${bolo.massaId}">Massa</label>
        <select id="${bolo.massaId}">
          <option value="Baunilha">Baunilha</option>
          <option value="Chocolate">Chocolate</option>
        </select>
      </div>

      <div class="field">
        <label for="${bolo.id}">Kg</label>
        <input type="number" id="${bolo.id}" min="0" step="0.1" value="0" />
      </div>
    </div>
  </div>
 </article>
`).join("");

elements.simplesGrid.innerHTML = bolosSimples.map((bolo) => `
 <article class="produto-card">
  <div class="produto-topo">
    <h4>${bolo.nome}</h4>
  </div>
  <div class="produto-meta">R$ ${formatarMoeda(bolo.preco)} por unidade</div>

  <div class="produto-actions">
    <div class="field">
      <label for="${bolo.id}">Quantidade</label>
      <input type="number" id="${bolo.id}" min="0" step="1" value="0" />
    </div>
  </div>
 </article>
`).join("");
```

```
}

function atualizarStatusBrigadeiro() {
  const status = elements.brigadeiroStatus;
  const { quantidade, invalido } = obterInfoBrigadeiro();
  const { total } = obterDistribuicaoBrigadeiros();

  status.className = "status-box";

  if (!quantidade) {
    status.classList.add("status-neutral");
    status.textContent = "Informe a quantidade total para começar a distribuir os sabores.";
    return;
  }

  if (invalido) {
    status.classList.add("status-danger");
    status.textContent = "O mínimo para brigadeiros é 50 unidades.";
    return;
  }

  if (total === quantidade) {
    status.classList.add("status-success");
    status.textContent = `Quantidade preenchida corretamente: ${total}/${quantidade}.`;
    return;
  }

  if (total < quantidade) {
    status.classList.add("status-warning");
    status.textContent = `Faltam ${quantidade - total} brigadeiros para completar o pedido.`;
    return;
```

```
  }

  status.classList.add("status-danger");
  status.textContent = `Você passou ${total - quantidade} brigadeiros do total informado.`;
}

function atualizarErroPagamento() {
  if (getFormaPagamento()) {
    elements.pagamentoErro.classList.add("hidden");
  }
}

function limparErrosCampos() {
  document.querySelectorAll(".erro-campo").forEach((el) => {
    el.classList.remove("erro-campo");
  });
}

function marcarErroCampo(id) {
  const el = document.getElementById(id);
  if (el) {
    el.classList.add("erro-campo");
  }
}

function mostrarMensagem(texto, tipo = "danger") {
  elements.formMensagem.className = "form-message";
  elements.formMensagem.classList.remove("hidden");

  if (tipo === "success") {
    elements.formMensagem.classList.add("status-success");
```

```
  } else if (tipo === "warning") {
    elements.formMensagem.classList.add("status-warning");
  } else {
    elements.formMensagem.classList.add("status-danger");
  }

  elements.formMensagem.textContent = texto;
}

function esconderMensagem() {
  elements.formMensagem.className = "form-message hidden";
  elements.formMensagem.textContent = "";
}

function obterItensResumo() {
  const itens = [];

  bolosGourmets.forEach((bolo) => {
    const kg = parseFloat(document.getElementById(bolo.id).value) || 0;

    if (kg > 0) {
      const massa = document.getElementById(bolo.massaId).value;
      itens.push({
        titulo: bolo.nome,
        descricao: `${kg} kg • Massa: ${massa} • R$ ${formatarMoeda(kg * bolo.preco)}`
      });
    }
  });

  bolosSimples.forEach((bolo) => {
    const qtd = parseInt(document.getElementById(bolo.id).value, 10) || 0;
```

```
    if (qtd > 0) {
      itens.push({
        titulo: bolo.nome,
        descricao: `${qtd} unid. • R$ ${formatarMoeda(qtd * bolo.preco)}`
      });
    }
  });

  const brigadeiroInfo = obterInfoBrigadeiro();
  const brigadeiroDistribuicao = obterDistribuicaoBrigadeiros();

  if (brigadeiroInfo.quantidade > 0) {
    itens.push({
      titulo: "Brigadeiros",
      descricao: `${brigadeiroInfo.quantidade} unid. • Preenchido:
${brigadeiroDistribuicao.total}/${brigadeiroInfo.quantidade} • R$
${formatarMoeda(brigadeiroInfo.preco)}`
    });
  }

  return itens;
}

function atualizarResumoVisual(total, valorAgora, restante) {
  const itens = obterItensResumo();

  if (!itens.length) {
    elements.resumoVisual.innerHTML = `
      <div class="resumo-vazio">
        Seu pedido ainda está vazio.<br />
        Escolha bolos ou brigadeiros para ver o resumo aqui.
```

```
      </div>
    `;
    return;
  }

  const condicaoPagamento = getCondicaoPagamento();
  const formaRecebimento = getFormaRecebimento();

  elements.resumoVisual.innerHTML = `
    <div class="resumo-item">
      <strong>Forma de recebimento</strong>
      <span>${formaRecebimento}</span>
    </div>

    ${itens.map((item) => `
      <div class="resumo-item">
        <strong>${item.titulo}</strong>
        <span>${item.descricao}</span>
      </div>
    `).join("")}

    <div class="resumo-item">
      <div class="resumo-total-linha"><span>Total do pedido</span><span>R$
${formatarMoeda(total)}</span></div>
      <div class="resumo-total-linha" style="margin-top: 8px;">
        <span>${condicaoPagamento === "avista" ? "Pagamento à vista" : "Sinal (50%)"}</span>
        <span>R$ ${formatarMoeda(valorAgora)}</span>
      </div>
      ${condicaoPagamento === "sinal" ? `
        <div class="resumo-total-linha" style="margin-top: 8px;">
          <span>Restante no recebimento</span>
```

```
        <span>R$ ${formatarMoeda(restante)}</span>
      </div>
    ` : ""}
  </div>
`;
}

function calcularTotal() {
  let total = 0;

  bolosGourmets.forEach((bolo) => {
    total += calcularSubtotalBoloGourmet(bolo.id, bolo.preco);
  });

  bolosSimples.forEach((bolo) => {
    total += calcularSubtotalBoloSimples(bolo.id, bolo.preco);
  });

  const brigadeiroInfo = obterInfoBrigadeiro();
  total += brigadeiroInfo.preco;

  const condicaoPagamento = getCondicaoPagamento();
  const formaRecebimento = getFormaRecebimento();

  let valorAgora = total;
  let restante = 0;

  if (condicaoPagamento === "sinal") {
    valorAgora = total / 2;
    restante = total / 2;
  }
```

```
    elements.total.textContent = formatarMoeda(total);

    elements.resumoRecebimentoTexto.textContent =

      `${formaRecebimento} • Pedidos com antecedência mínima de 1 dia • Horário entre 07:00
e 20:00`;


    if (condicaoPagamento === "avista") {

      elements.linhaEntrada.innerHTML = `Pagar agora: R$ <span
id="sinal">${formatarMoeda(valorAgora)}</span>`;

      elements.linhaRestante.classList.add("hidden");

      elements.textoCondicaoPagamento.textContent = "Pagamento à vista no momento do
pedido.";

    } else {

      elements.linhaEntrada.innerHTML = `Sinal (50%): R$ <span
id="sinal">${formatarMoeda(valorAgora)}</span>`;

      elements.linhaRestante.innerHTML = `Restante no recebimento: R$ <span
id="restante">${formatarMoeda(restante)}</span>`;

      elements.linhaRestante.classList.remove("hidden");

      elements.textoCondicaoPagamento.textContent =

        "Pagamento com 50% de sinal no momento do pedido e 50% no recebimento.";

    }


    const botao = elements.form.querySelector('button[type="submit"]');

    botao.disabled = total <= 0;


    atualizarStatusBrigadeiro();

    atualizarResumoVisual(total, valorAgora, restante);

  }


  function montarResumoTexto() {

    let resumo = "Resumo do pedido:\n\n";

    let temItem = false;
```

```
    bolosGourmets.forEach((bolo) => {
      const kg = parseFloat(document.getElementById(bolo.id).value) || 0;

      if (kg > 0) {
        const massa = document.getElementById(bolo.massaId).value;
        const subtotal = kg * bolo.preco;
        resumo += `Bolo gourmet: ${bolo.nome} | Massa: ${massa} | ${kg}kg | R$
${formatarMoeda(subtotal)}\n`;
        temItem = true;
      }
    });

    bolosSimples.forEach((bolo) => {
      const quantidade = parseInt(document.getElementById(bolo.id).value, 10) || 0;

      if (quantidade > 0) {
        const subtotal = quantidade * bolo.preco;
        resumo += `Bolo simples: ${bolo.nome} | ${quantidade} unid. | R$
${formatarMoeda(subtotal)}\n`;
        temItem = true;
      }
    });

    const brigadeiroInfo = obterInfoBrigadeiro();
    const brigadeiroDistribuicao = obterDistribuicaoBrigadeiros();

    if (brigadeiroInfo.quantidade > 0) {
      const saboresTexto = brigadeiroDistribuicao.sabores.length
        ? brigadeiroDistribuicao.sabores.map((item) => `${item.sabor}:
${item.quantidade}`).join(", ")
        : "Não selecionado";
```

```
  resumo += `Brigadeiros: ${brigadeiroInfo.quantidade} unidades\n`;
  resumo += `Sabores: ${saboresTexto}\n`;
  resumo += `Preenchido: ${brigadeiroDistribuicao.total}/${brigadeiroInfo.quantidade}\n`;
  resumo += `Valor dos brigadeiros: R$ ${formatarMoeda(brigadeiroInfo.preco)}\n`;
  temItem = true;
}

if (!temItem) {
  resumo += "Nenhum item selecionado.\n";
}

const nome = elements.nome.value.trim() || "-";
const email = elements.email.value.trim() || "-";
const cep = elements.cep.value.trim() || "-";
const endereco = elements.endereco.value.trim() || "-";
const numero = elements.numero.value.trim() || "-";
const complemento = elements.complemento.value.trim() || "-";
const data = elements.dataPedido.value || "-";
const hora = elements.horaPedido.value || "-";
const observacoes = elements.observacoes.value.trim() || "-";
const formaPagamento = getFormaPagamento() || "-";
const formaRecebimento = getFormaRecebimento();
const total = elements.total.textContent;
const sinalEl = document.getElementById("sinal");
const restanteEl = document.getElementById("restante");
const sinal = sinalEl ? sinalEl.textContent : "0,00";
const restante = restanteEl ? restanteEl.textContent : "0,00";
const condicaoPagamento = getCondicaoPagamento();

resumo += `\nNome: ${nome}\n`;
resumo += `E-mail: ${email}\n`;
```

```
    resumo += `Forma de recebimento: ${formaRecebimento}\n`;
    resumo += `CEP: ${cep}\n`;
    resumo += `Endereço: ${endereco}\n`;
    resumo += `Número: ${numero}\n`;
    resumo += `Complemento: ${complemento}\n`;
    resumo += `Data: ${data}\n`;
    resumo += `Hora: ${hora}\n`;
    resumo += `Forma de pagamento: ${formaPagamento}\n`;
    resumo += `Observações: ${observacoes}\n`;

    if (condicaoPagamento === "avista") {
      resumo += "Condição de pagamento: À vista.\n";
    } else {
      resumo += "Condição de pagamento: 50% de sinal no momento do pedido e 50% no
recebimento.\n";
    }

    resumo += `\nTotal: R$ ${total}\n`;

    if (condicaoPagamento === "avista") {
      resumo += `Pagamento à vista: R$ ${sinal}`;
    } else {
      resumo += `Sinal (50%): R$ ${sinal}\n`;
      resumo += `Restante no recebimento (50%): R$ ${restante}`;
    }

    return resumo;
  }

  function resetarFormulario() {
    elements.form.reset();
```

```
    document.querySelectorAll(".brigadeiro-qtd").forEach((input) => {
      input.value = 0;
    });

    elements.pagamentoErro.classList.add("hidden");

    const novaDataMinima = getDataMinima();
    elements.dataPedido.min = novaDataMinima;
    elements.dataPedido.value = novaDataMinima;
    elements.horaPedido.value = "07:00";

    const radioCondicao =
document.querySelector('input[name="condicaoPagamento"][value="avista"]');
    if (radioCondicao) {
      radioCondicao.checked = true;
    }

    const radioRecebimento =
document.querySelector('input[name="recebimento"][value="Retirada"]');
    if (radioRecebimento) {
      radioRecebimento.checked = true;
    }
  }

  function limparCEP(valor) {
    return valor.replace(/\D/g, "");
  }

  function formatarCEP(valor) {
    const cepLimpo = limparCEP(valor);
```

```
  if (cepLimpo.length <= 5) return cepLimpo;
  return `${cepLimpo.slice(0, 5)}-${cepLimpo.slice(5, 8)}`;
}

async function buscarCEP(cep) {
  const cepLimpo = limparCEP(cep);

  if (cepLimpo.length !== 8) return;

  try {
    elements.endereco.value = "Buscando endereço...";

    const response = await fetch(`https://viacep.com.br/ws/${cepLimpo}/json/`);
    const data = await response.json();

    if (data.erro) {
      elements.endereco.value = "";
      mostrarMensagem("CEP não encontrado.", "warning");
      return;
    }

    elements.endereco.value = [data.logradouro, data.bairro, data.localidade, data.uf]
      .filter(Boolean)
      .join(" - ");

    if (!elements.complemento.value && data.complemento) {
      elements.complemento.value = data.complemento;
    }
  } catch (error) {
    elements.endereco.value = "";
    mostrarMensagem("Não foi possível buscar o endereço pelo CEP.", "warning");
```

```
  }
}

async function enviarPedido(event) {
  event.preventDefault();
  limparErrosCampos();
  esconderMensagem();

  const botao = elements.form.querySelector('button[type="submit"]');
  const nome = elements.nome.value.trim();
  const email = elements.email.value.trim();
  const data = elements.dataPedido.value;
  const hora = elements.horaPedido.value;
  const formaPagamento = getFormaPagamento();
  const formaRecebimento = getFormaRecebimento();
  const cep = elements.cep.value.trim();
  const endereco = elements.endereco.value.trim();
  const numero = elements.numero.value.trim();
  const totalTexto = elements.total.textContent;
  const totalNumero = Number(totalTexto.replace(",", "."));
  const dataMinima = getDataMinima();
  const resumo = montarResumoTexto();

  if (totalNumero <= 0) {
    mostrarMensagem("Selecione pelo menos um item para fazer o pedido.", "danger");
    return;
  }

  if (nome.length < 3) {
    marcarErroCampo("nome");
    mostrarMensagem("Informe um nome válido com pelo menos 3 caracteres.", "danger");
```

```
    return;
  }

  if (!emailValido(email)) {
    marcarErroCampo("email");
    mostrarMensagem("Informe um e-mail válido.", "danger");
    return;
  }

  if (!data) {
    marcarErroCampo("dataPedido");
    mostrarMensagem("Selecione a data.", "danger");
    return;
  }

  if (!hora) {
    marcarErroCampo("horaPedido");
    mostrarMensagem("Selecione o horário.", "danger");
    return;
  }

  if (data < dataMinima) {
    marcarErroCampo("dataPedido");
    const dataFormatada = dataMinima.split("-").reverse().join("/");
    mostrarMensagem(`Os pedidos devem ser feitos com 1 dia de antecedência. Data mínima: ${dataFormatada}.`, "danger");
    return;
  }

  if (hora < "07:00" || hora > "20:00") {
    marcarErroCampo("horaPedido");
```

```
  mostrarMensagem("O horário deve estar entre 07:00 e 20:00.", "danger");
  return;
}

if (!formaPagamento) {
  elements.pagamentoErro.classList.remove("hidden");
  mostrarMensagem("Selecione a forma de pagamento.", "danger");
  return;
}

if (formaRecebimento === "Entrega") {
  if (!cep || limparCEP(cep).length !== 8) {
    marcarErroCampo("cep");
    mostrarMensagem("Informe um CEP válido para entrega.", "danger");
    return;
  }

  if (!endereco) {
    marcarErroCampo("endereco");
    mostrarMensagem("Informe o endereço para entrega.", "danger");
    return;
  }

  if (!numero) {
    marcarErroCampo("numero");
    mostrarMensagem("Informe o número do endereço para entrega.", "danger");
    return;
  }
}

const brigadeiroInfo = obterInfoBrigadeiro();
```

```
    const brigadeiroDistribuicao = obterDistribuicaoBrigadeiros();

    if (brigadeiroInfo.invalido) {
      marcarErroCampo("brigadeiroQtd");
      mostrarMensagem("O mínimo para brigadeiros é 50 unidades.", "danger");
      return;
    }

    if (
      brigadeiroInfo.quantidade > 0 &&
      brigadeiroDistribuicao.total !== brigadeiroInfo.quantidade
    ) {
      marcarErroCampo("brigadeiroQtd");
      mostrarMensagem(`A soma dos brigadeiros precisa ser exatamente
${brigadeiroInfo.quantidade} unidades.`, "danger");
      return;
    }

    try {
      botao.disabled = true;
      botao.textContent = "Enviando...";

      const response = await fetch(FORMSPREE_URL, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          "Accept": "application/json"
        },
        body: JSON.stringify({
          nome,
          email,
```

```
        forma_recebimento: formaRecebimento,
        cep: elements.cep.value.trim(),
        endereco: elements.endereco.value.trim(),
        numero: elements.numero.value.trim(),
        complemento: elements.complemento.value.trim(),
        data_pedido: data,
        hora_pedido: hora,
        observacoes: elements.observacoes.value.trim(),
        forma_pagamento: formaPagamento,
        condicao_pagamento: getCondicaoPagamento(),
        total: `R$ ${totalTexto}`,
        resumo_pedido: resumo
      })
    });

    if (!response.ok) {
      throw new Error("Falha ao enviar");
    }

    mostrarMensagem("Pedido enviado com sucesso! Entraremos em contato se necessário.",
"success");
    resetarFormulario();
    calcularTotal();
  } catch (error) {
    mostrarMensagem("Não foi possível enviar o pedido. Verifique sua conexão e tente
novamente.", "danger");
  } finally {
    botao.disabled = false;
    botao.textContent = "Enviar Pedido";
    calcularTotal();
  }
}
```

```
function inicializarFormulario() {
  renderizarProdutos();

  const dataMinimaInicial = getDataMinima();
  elements.dataPedido.min = dataMinimaInicial;
  elements.dataPedido.value = dataMinimaInicial;

  elements.form.addEventListener("submit", enviarPedido);

  elements.cep.addEventListener("input", (event) => {
    event.target.value = formatarCEP(event.target.value);
  });

  elements.cep.addEventListener("blur", () => {
    buscarCEP(elements.cep.value);
  });

  document.addEventListener("input", (event) => {
    const target = event.target;

    if (
      target.matches("input[type='number'], input[type='text'], input[type='email'],
input[type='date'], input[type='time'], textarea, select")
    ) {
      if (target.value !== "") {
        target.classList.remove("erro-campo");
      }

      calcularTotal();
    }
```

```
    });

    document.addEventListener("change", (event) => {
      const target = event.target;

      if (target.matches("input[name='pagamento']")) {
        atualizarErroPagamento();
      }

      if (target.matches("input, select, textarea")) {
        calcularTotal();
      }
    });

    calcularTotal();
  }

  inicializarFormulario();
 </script>
</body>
</html>
```